

# Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez,  
Łukasz Kaiser, Illia Polosukhin

## ***Abstract***

*The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train.*

## **1. Introduction**

Recurrent neural networks, especially long short-term memory (LSTM) and gated recurrent (GRU) neural networks, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation. Subsequent efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures.

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states  $h_t$ , as a function of the previous hidden state  $h_{t-1}$  and the input for position  $t$ . This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths.

The primary contribution of this work is the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

## 2. Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure. Here, the encoder maps an input sequence of symbol representations  $(x_1, \dots, x_n)$  to a sequence of continuous representations  $z = (z_1, \dots, z_n)$ . Given  $z$ , the decoder then generates an output sequence  $(y_1, \dots, y_m)$  of symbols one element at a time. At each step the model is auto-regressive, consuming the previously generated symbols as additional input when generating the next.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder. The encoder is composed of a stack of  $N = 6$  identical layers. Each layer has two sub-layers: a multi-head self-attention mechanism and a simple, position-wise fully connected feed-forward network. We employ a residual connection around each of the two sub-layers, followed by layer normalization.

## 3. Multi-Head Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

Instead of performing a single attention function with  $d_{\text{model}}$ -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values  $h$  times with different, learned linear projections to  $d_k$ ,  $d_k$  and  $d_v$  dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding  $d_v$ -dimensional output values.

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this. In our work we employ  $h = 8$  parallel attention layers, or heads. For each of these we use  $d_k = d_v = d_{\text{model}} / h = 64$ .

## 4. Positional Encoding and Training

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add positional encodings to the input embeddings at the bottoms of the encoder and decoder stacks.

The positional encodings have the same dimension  $d_{\text{model}}$  as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed. In this work, we use sine and cosine functions of different frequencies:  $PE(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i / d_{\text{model}})})$  and  $PE(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i / d_{\text{model}})})$ .

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. On the WMT 2014 English-to-German translation task, the big transformer model establishes a new state-of-the-art BLEU score of 28.4.